

一、作業題目

這次作業目的是實作色彩空間轉換演算法，將影像從 RGB 轉換至 HSI (Hue, Saturation, Intensity) 空間，並針對三個通道分別進行影像增強處理，最後還原回 RGB 空間以觀察結果。

核心要求：

1. **影像 I/O 與格式處理 (R1)：**讀取 Sequential (Planar) 格式的 Raw 檔 (512×512)，並能夠正確處理非標準尺寸的測試檔案（如 1.raw, 2.raw）。
2. **色彩空間轉換 (R2)：**實作 RGB 與 HSI 的雙向轉換公式，過程需使用浮點數運算以確保精度，並處理還原時的 Overflow 問題。
3. **通道增強處理 (R3)：**
 - I Channel：以直方圖等化 (Histogram Equalization) 進行亮度對比加強。
 - S Channel：以 Gamma 轉換 (Gamma Transformation) 進行飽和度加強。
 - H Channel：以自行定義的方法（如色相偏移）進行色調變化。
4. **結果驗證 (R4)：**以 baboon.raw, lena.raw 及其他額外檔案進行測試，並比較處理前後的差異。

二、實作方法

本次作業使用 Python 與 NumPy 函式庫作為核心數值計算工具。整體架構遵循「讀取 → 轉換 → 處理 → 還原 → 寫入」的流程。

2.1 程式設計思路與 Sequential Raw 處理

A. Sequential (Planar) 格式讀取機制：為了滿足作業要求的 Sequential 格式（資料排列為整張 R 平面 → G 平面 → B 平面），程式不使用標準影像庫讀取，而是直接操作 Binary Stream。

- 使用 `numpy.fromfile` 讀取 `uint8` 數據。
- 透過 `reshape(3, H, W)` 與 `transpose(1, 2, 0)` 將資料重組為 (Height, Width, Channel) 格式。

B. 前處理 (Pre-processing)：針對測試檔 1.raw 與 2.raw 檔案大小不符 $512 \times 512 \times 3$ 的問題，實作了一個前處理腳本。該腳本能自動偵測並讀取含有檔頭的圖片，將其強制縮放至標準尺寸並轉存為純二進位的 Sequential Raw 格式，確保主程式能順利執行。

2.2 色彩空間轉換數學實作細節

所有轉換運算皆強制轉型為 float64 以避免截斷誤差。

- RGB $\rightarrow$ HSI：

- $I = (R + G + B)/3$ 。

- $S = 1 - \frac{3}{R+G+B} [\min(R, G, B)]$ ，程式碼中加入了分母為零的保護機制。

- H：

$$\theta = \cos^{-1} \left\{ \frac{\frac{1}{2}[(R-G) + (R-B)]}{\left[(R-G)^2 + (R-B)(G-B) \right]^{1/2}} \right\}$$

$$H = \begin{cases} \theta & \text{if } B \leq G \\ 360 - \theta & \text{if } B > G \end{cases}$$

- HSI $\rightarrow$ RGB：依據 H 的角度區間

甲、 $0^\circ \leq H < 120^\circ$ ：

If $0^\circ \leq H < 120^\circ$,

$$R = I \left[1 + \frac{S \cos H}{\cos(60^\circ - H)} \right]$$

$$G = 3I - (R + B)$$

$$B = I(1 - S)$$

乙、 $120^\circ \leq H < 240^\circ$ ：

If $120^\circ \leq H < 240^\circ$, $H = H - 120$

$$R = I(1 - S)$$

$$G = I \left[1 + \frac{S \cos H}{\cos(60^\circ - H)} \right]$$

$$B = 3I - (R + G)$$

丙、 $240^\circ \leq H < 360^\circ$:

If $240^\circ \leq H \leq 360^\circ$, $H = H - 240$

$$R = 3I - (G + B)$$

$$G = I(1 - S)$$

$$B = I \left[1 + \frac{S \cos H}{\cos(60^\circ - H)} \right]$$

利用 Boolean Masking 分段套用還原公式。

- **溢位保護**：還原後的 RGB 數值可能超出 $[0, 255]$ ，使用 $\min(255, \max(0, val))$ 進行截斷，防止二進位溢位造成的噪點。

2.3 通道增強演算法實作

- **I Channel (HE)**：計算亮度值的累積機率分佈 (CDF)，建立映射表將亮度重新分配，使直方圖趨於均勻。
- **S Channel (Gamma)**：使用 $S_{new} = S_{old}^{0.6}$ 。由於 $\gamma < 1$ ，曲線呈上凸，能有效提升低飽和度區域的數值，使色彩更鮮豔。
- **H Channel (Shift)**：採用 $H_{new} = (H_{old} + 30)(mod\ 360)$ 。將色相環整體旋轉 30 度，使紅色偏移至橙黃色，藍色偏移至紫色，創造風格化濾鏡效果。

三、工作進度

階段	實作內容	驗證與結果
P1	Sequential Raw I/O (R1)	成功讀取 baboon.raw 和 lena.raw，並確認 RGB 平面重組正確，無影像錯位。
P2	RGB $\Leftrightarrow$ HSI 轉換核心 (R2)	實作雙向轉換公式，驗證還原後的 RGB 影像與原圖一致。
P3	前處理工具開發	針對 1.raw 和 2.raw 讀取錯誤問題，開發轉檔工具，成功將其修正為標準格式。

P4	通道增強演算法 (R3)	實作 HE、Gamma 與 Hue Shift，並分別觀察各通道處理後的視覺變化。
P5	整合測試與報告	處理四張測試圖檔，生成對照圖表，確認所有演算法疊加後的綜合效果。

四、 結果測試

本節針對四張測試圖檔 (baboon, lena, 1_fixed, 2_fixed) 展示完整的 HSI 通道分離測試結果。每組圖片從左至右依序為：原始圖、僅 H 變化、僅 S 變化、僅 I 變化。

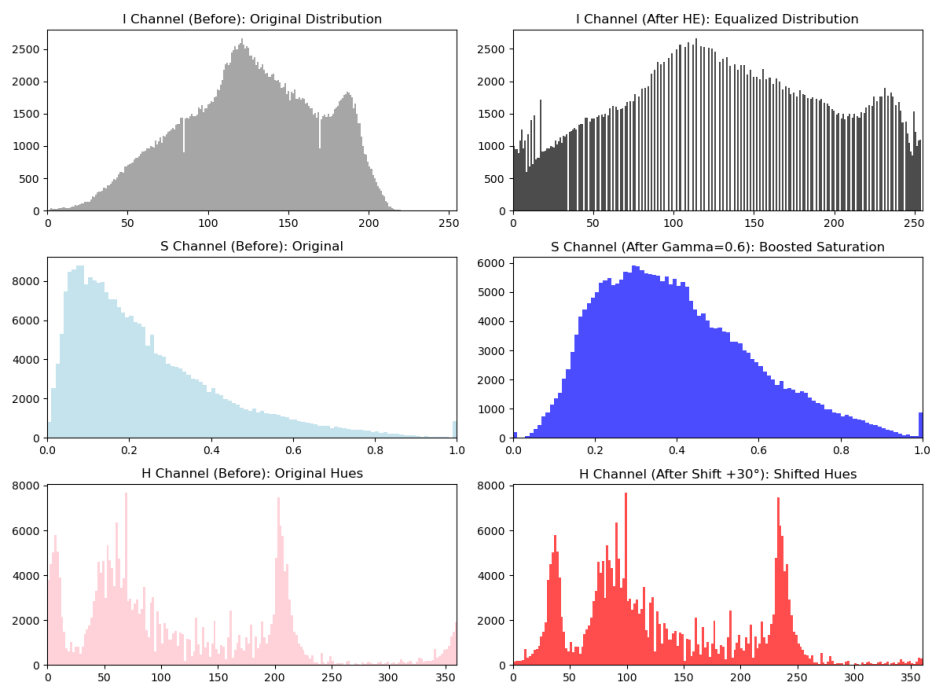
4.1 測試一：baboon



(圖 1：baboon 影像之單通道分離測試)

- H Channel Only：觀察 baboon 的鼻子，原本為鮮紅色 (0°)，經 $+30^\circ$ 偏移後精確轉變為金黃色，且亮度與飽和度完全未受影響，證明 H 通道獨立性。
- S Channel Only：Gamma 校正後，臉部藍色與鼻子的紅色變得極度鮮豔飽和。
- I Channel Only：毛髮細節在直方圖等化後變得非常銳利，但顏色變淡。

Channel Histogram Analysis: baboon.raw



(圖 2：baboon 影像處理前後之 HSI 通道直方圖分析)

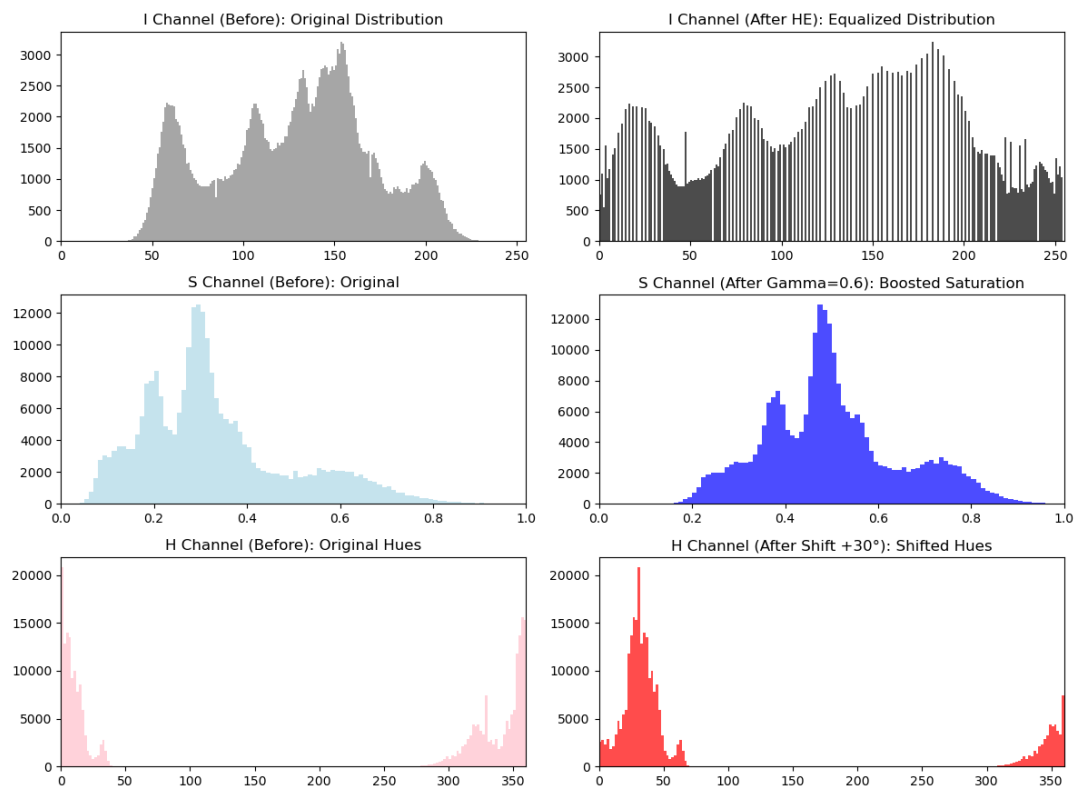
4.2 測試二：lena



(圖 3：lena 影像之單通道分離測試)

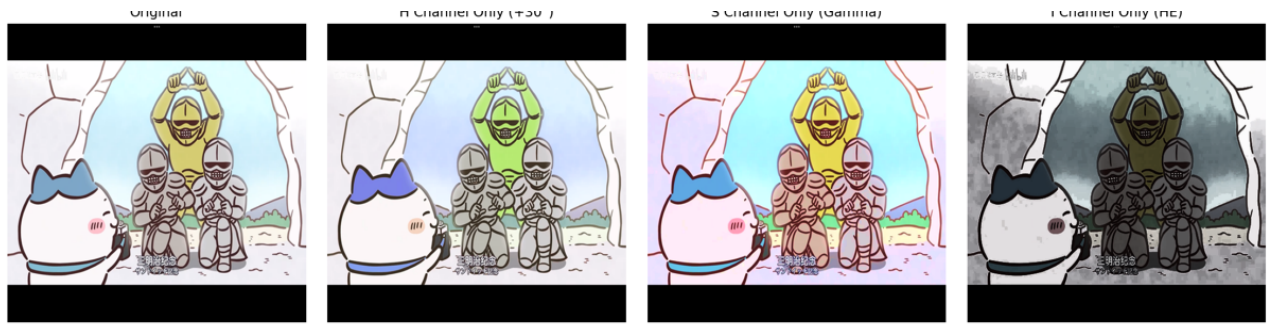
- **H Channel Only**：lena 的膚色與粉紅帽子轉變為偏黃暖色調，展示了色相偏移對人像攝影的风格影響。
- **S Channel Only**：帽子的羽毛裝飾色彩變得更加鮮豔，細節層次更豐富。
- **I Channel Only**：臉部陰影與背景柱子的暗部細節被拉亮，對比度顯著提升。

Channel Histogram Analysis: lena.raw



(圖 4：lena 影像處理前後之 HSI 通道直方圖分析)

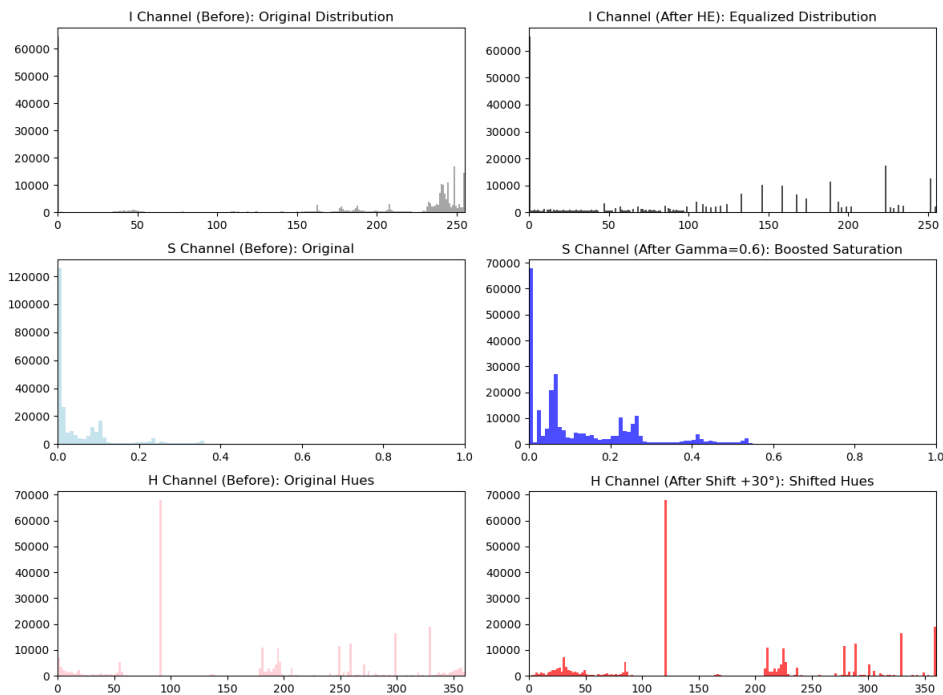
4.3 測試三：1_fixed



(圖 5：1_fixed 影像之單通道分離測試)

- **H Channel Only**：這張圖最能展現色相旋轉的效果。原本藍色的蝴蝶結（ 240° ）變成了紫色（ 270° ）；鎧甲人原為鮮黃色（ 60° ），經 $+30^\circ$ 偏移後，轉變為鮮綠色（ 90° ）。由大面積純色塊組成，色相的精確平移驗證了演算法對不同色域的處理皆正確無誤。
- **S Channel Only**：圖片整體的鮮豔度提升，消除了原圖較為灰暗的感覺。
- **I Channel Only**：整體畫面亮度分布更加均勻，但在大色塊區域並未造成過度曝光。

Channel Histogram Analysis: 1_fixed.raw



(圖 6：1_fixed 影像處理前後之 HSI 通道直方圖分析)

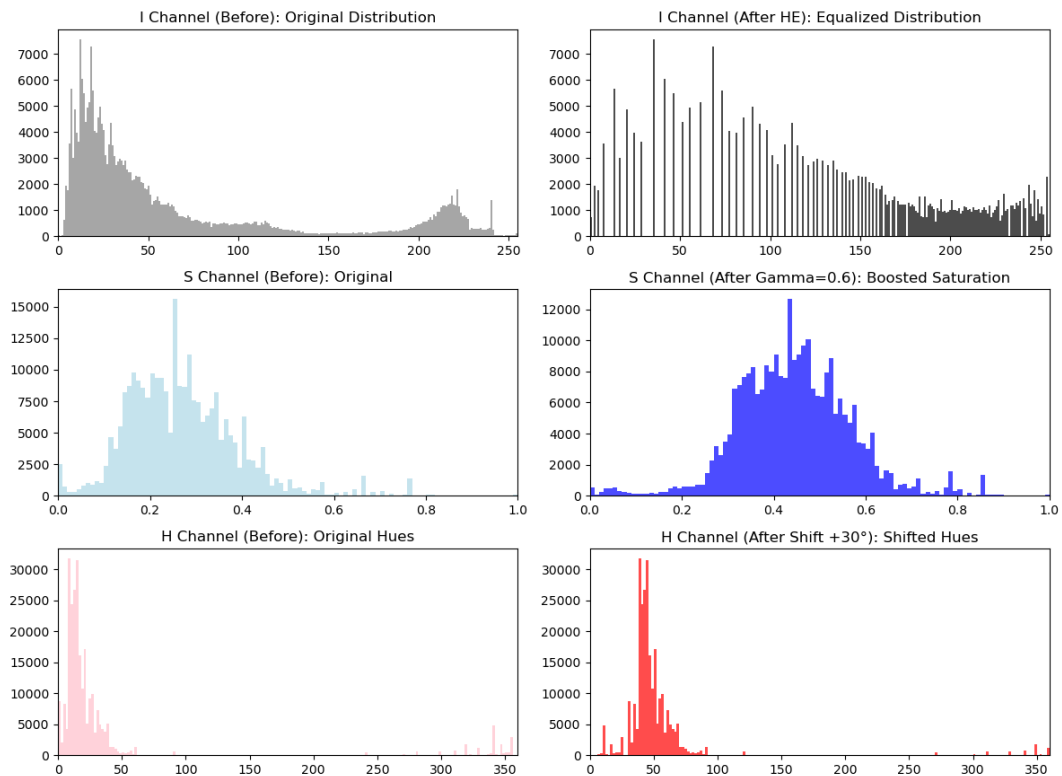
4.4 測試四：2_fixed



(圖 7：2_fixed 影像之單通道分離測試)

- H Channel Only：整體的暖色調氛圍進一步增強，黃色光線轉向略帶金橙色的色調。
- S Channel Only：觀察背景的黃色光暈，Gamma 增強後光影效果變得極具張力，視覺衝擊感大幅提升。
- I Channel Only：原本左側較暗的背景細節，在 HE 處理後完全浮現。

Channel Histogram Analysis: 2_fixed.raw



(圖 8：2_fixed 影像處理前後之 HSI 通道直方圖分析)

五、心得報告

本次作業不僅是一次程式實作的練習，更是一次對數位影像底層原理的深刻探索。在完成作業的過程中，我有三個主要的學習與體悟。

首先是對「資料格式」的敬畏。在實作初期，我理所當然的認為所有 .raw 檔都符合規範，直到 1.raw 讀取出一片混亂的雪花時，我才意識到真實世界的資料往往是「髒」的。一開始我是直接將副檔名改成 raw 檔，發現錯誤後改成撰寫前處理腳本將其轉為標準 Sequential 格式，這個 Debug 的過程讓我學到了比演算法本身更寶貴的經驗：資料清理 (Data Cleaning) 往往是多媒體處理最關鍵的第一步。此外，我也深刻理解了 Planar (RRR...GGG...) 與 Interleaved (RGBRGB...) 排列在記憶體存取上的差異，這對於未來處理大型影像資料的效能優化至關重要。

其次是數學公式與視覺效果的連結。過去在課本上看到 RGB 轉 HSI 的複雜三角函數公式時，只覺得枯燥乏味。然而，當我親手將這些公式轉化為 Python 程式碼，並看著 Baboon 的紅鼻子因為簡單的「矩陣加法 (H+30)」而瞬間變成金黃色時，那種「數學落地」的成就感油然而生。我深刻體會到 HSI 模型的強大之處在於它將「色彩資訊 (Chrominance)」與「亮度資訊 (Luminance)」分離。在 RGB 模式下，若想調整亮度而不改變顏色是非常困難的；但在 HSI 模式下，我們可以直接對 I Channel 做直方圖等化，完全不用擔心顏色會跑掉，這讓我真正理解了為何 HSI 在影像處理領域如此重要。

最後是工程實作的細節打磨。在實作過程中，我遇到了「除以零」與「數值溢位 (Overflow)」等陷阱。例如在計算 Saturation 時，若 $R+G+B=0$ 會導致程式崩潰；又或者在轉回 RGB 時，數值超過 255 會導致畫面出現詭異的黑點。透過引入 float64 進行高精度運算，並在最後輸出階段加入 Clipping 機制，我學會了如何寫出更強健的程式碼。

總結來說，這次作業讓我從資料面、數學面到實作面都有了全面的提升。它不只是一個作業，更讓我具備了基礎的影像處理引擎開發能力，我非常享受這次「玩轉色彩」的過程。

六、 參考資料

- 課程講義: HW2_HSI_2025.pdf, Raw_Data_Format.pdf
- [Python HSI to RGB conversion - not what I expect](#)
- [\[OpenCV 基礎\]\[Python\]影像增強_直方圖均衡處理](#)
- [\[OpenCV 基礎\]\[Python\]影像增強_伽瑪校正 Gamma correction](#)
- AI 詢問(Google Gemini)